

BFS接近法 and Code

This problem is a **BFS / shortest path** problem.

1. Core intuition

Think of every lock combination as a **node in a graph**.

For example:

```
0000
```

From **0000**, in **one turn**, we can reach 8 states:

```
1000  9000
0100  0900
0010  0090
0001  0009
```

Why 8?

There are 4 wheels, and for each wheel we can turn:

- +1
- -1

So:

```
4 wheels × 2 directions = 8 neighbors
```

Therefore, the problem becomes:

Starting from **0000**, find the shortest number of edges needed to reach **target**, while never entering a deadend.

And **BFS is exactly for finding the shortest path in an unweighted graph**.

2. Why BFS?

Suppose:

```
0000
```

BFS explores like this:

```
distance 0:
0000

distance 1:
1000, 9000, 0100, 0900, ...

distance 2:
all states reachable in 2 turns

distance 3:
all states reachable in 3 turns

...
```

So the **first time we reach target**, we have found the minimum number of turns.

3. Important part: generating neighbors

Suppose current state is:

```
"1234"
```

For the first wheel:

```
1234 -> 2234  // +1
1234 -> 0234  // -1
```

For second:

```
1234 -> 1334
1234 -> 1134
```

And so on.

Wrap-around

If digit is 9:

```
9 + 1 = 0
```

If digit is 0:

```
0 - 1 = 9
```

We can handle this easily:

```
C++
char up = (state[i] - '0' + 1) % 10 + '0';
char down = (state[i] - '0' + 9) % 10 + '0';
```

Why +9?

Because:

```
(digit - 1 + 10) % 10
```

is the same as:

```
(digit + 9) % 10
```

4. Deadends and visited

We need two things:

dead

A set containing states we are **not allowed to enter**.

```
C++
unordered_set<string> dead;
```

visited

Without visited, BFS could keep going in circles:

```
0000
1000
0000
1000
...
```

So whenever we add a state to the queue, mark it visited.

```
C++
visited.insert(next);
```

5. Algorithm

1. Put all deadends into a set.
2. If "0000" itself is a deadend:
return -1
3. Start BFS from "0000".
4. For every state:
Generate 8 possible next states.
5. Ignore a state if:
 - it is a deadend
 - it was already visited
6. If next state == target:
return current distance + 1
7. If BFS finishes:
return -1

6. C++ Code

```
C++
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int openLock(vector<string>& deadends, string target) {

        // Store all deadend states
        unordered_set<string> dead;

        for (string s : deadends) {
            dead.insert(s);
        }

        // If starting position itself is blocked
        if (dead.count("0000")) {
            return -1;
        }

        // BFS queue: {current state, number of moves}
        queue<pair<string, int>> q;

        // To avoid visiting the same state again
        unordered_set<string> visited;

        q.push({"0000", 0});
        visited.insert("0000");

        while (!q.empty()) {

            string curr = q.front().first;
            int moves = q.front().second;
            q.pop();

            // We reached the target
            if (curr == target) {
                return moves;
            }

            // Try changing each of the 4 wheels
            for (int i = 0; i < 4; i++) {

                // Move wheel forward (+1)
                string next = curr;

                next[i] = (curr[i] - '0' + 1) % 10 + '0';

                if (!dead.count(next) && !visited.count(next)) {
                    visited.insert(next);
                    q.push({next, moves + 1});
                }

                // Move wheel backward (-1)
                next = curr;

                next[i] = (curr[i] - '0' + 9) % 10 + '0';

                if (!dead.count(next) && !visited.count(next)) {
                    visited.insert(next);
                    q.push({next, moves + 1});
                }
            }
        }

        // Target cannot be reached
        return -1;
    }
};
```

7. Understand the important loop

This is the heart of the solution:

```
C++
for (int i = 0; i < 4; i++) {
```

We select one wheel.

Then:

```
C++
next[i] = (curr[i] - '0' + 1) % 10 + '0';
```

means **turn that wheel forward**.

And:

```
C++
next[i] = (curr[i] - '0' + 9) % 10 + '0';
```

means **turn that wheel backward**.

Therefore each state produces:

4 wheels × 2 directions
 ↓
 8 states

Then BFS processes those states level by level.

Complexity

There are only:

$10 \times 10 \times 10 \times 10 = 10,000$

possible lock combinations.

Each state generates only 8 neighbors.

So:

Time: $O(10,000 \times 8) \rightarrow$ effectively $O(1)$ for this fixed-size problem.

Space: $O(10,000)$ for the queue/visited/deadend sets.

The main pattern to remember is:

When a problem asks for minimum moves/steps and every move has equal cost $\rightarrow$ think BFS.

ChatGPT can make mistakes. Check important info.